

BTI 3021: Networking Project - Sprint 1

Christian Grothoff

1 Introduction

For this sprint you will implement, document and test an Ethernet switch in userland under GNU/Linux.

While the driver and skeleton you are given is written in C, you may use *any* language of your choice for the implementation (as long as you extend the `Makefile` with adequate build rules). However, if you choose a different language, be prepared to write additional boilerplate yourselves.

How an Ethernet switch works is expected to be understood from the networking class. If not, you can find plenty of documentation and specifications on the Internet.

1.1 Deliverables

There will be three main deliverables for the sprint:

Documentation You must document the main data structure (switching table), the switching algorithm and a **comprehensive test plan**.

Implementation You must implement the switching algorithm, extending the `switch.c` template provided (or write the entire logic from scratch in another language).

Testing You must implement and submit your own **test cases** by *pretending* to be the network driver (see below) and sending various Ethernet frames to your program and verifying that it outputs the correct frames. Additionally, you should perform and **document** *interoperability* tests against existing implementations (i.e. other notebooks from your team to ensure that your switch integrates correctly with other implementations).

All deliverables must be submitted to Git (master branch) by **November 4th 2020, 18:00 CET**.

1.2 Team

You are expected to work in a team of four students. The suggested division of work is:

1 Student Scrum master, also responsible for the documentation

1 Student Implementation

2 Students Testing

1.3 Grading

You get points for each of the key deliverables:

Technical documentation (A&D, test plan)	3
Correct implementation	7
Comprehensive test cases	4
Quality of English in documentation	2
Total	16

We will specifically also look for the following properties of a switch:

- Support for multicast and broadcast
- Changing external connections (re-learning when devices move around the network)
- Managing an “attacker” process that sends from billions of MAC addresses. Ensure your switch’s learning table uses finite memory.

Partial points are awarded.

You can earn 16, 15 and 19 points in the three networking sprints, for a total of 50 points. You need **37 points** to pass the networking component of BTI 3021.

2 Hardware

You will be given a 4-port Ethernet USB adapter that you can use to add four physical ports to any PC. If you use the laboratory PCs, be aware that some of the USB ports provide insufficient power. Which ones work is inconsistent even across identical PCs and often even the adjacent USB port works even though it looks identical!

You are not expected to write a driver to interact directly with the Ethernet USB adapter. Instead, you will use the provided **network-driver** which can already provide you with raw access to any Ethernet interface (incl. WLAN).

2.1 Alternative setup with virtual machines

Download a basic qemu VM, for example from <https://people.debian.org/~aurel32/qemu/amd64/>.

Then launch your router with four network interfaces and the respective up scripts and disjoint MACs:

```
# qemu-system-x86_64 -hda wheezy.qcov2 \
  -device e1000,netdev=net0,mac=10:10:A0:D0:C0:B1 \
  -netdev socket,id=net0,listen=:10001 \
  -device e1000,netdev=net1,mac=10:10:A0:D0:C0:B2 \
  -netdev socket,id=net0,listen=:10002 \
  -device e1000,netdev=net2,mac=10:10:A0:D0:C0:B3 \
  -netdev socket,id=net0,listen=:10003 \
  -device e1000,netdev=net3,mac=10:10:A0:D0:C0:B4 \
  -netdev socket,id=net0,listen=:10004
```

Then, launch additional VMs to connect “other” systems to your router:

```
# cp wheezy.qcov2 wheezy0.qcov
# qemu-system-x86_64 -hda wheezy0.qcov2 \
  -device e1000,netdev=net0,mac=10:20:A0:D0:C0:B1 \
  -netdev socket,id=net0,connect:10001
# cp wheezy.qcov2 wheezy1.qcov
# qemu-system-x86_64 -hda wheezy1.qcov2 \
  -device e1000,netdev=net0,mac=10:20:A0:D0:C0:B2 \
  -netdev socket,id=net0,connect:10002
# cp wheezy.qcov2 wheezy2.qcov
# qemu-system-x86_64 -hda wheezy2.qcov2 \
  -device e1000,netdev=net0,mac=10:20:A0:D0:C0:B3 \
  -netdev socket,id=net0,connect:10003
# cp wheezy.qcov2 wheezy3.qcov
# qemu-system-x86_64 -hda wheezy3.qcov2 \
  -device e1000,netdev=net0,mac=10:20:A0:D0:C0:B4 \
  -netdev socket,id=net0,connect:10004
```

You may want to link up your VMs to your host to copy files:

```
# qemu-system-x86_64 ...
  -net nic -net user,hostfwd=tcp::5555-:22
```

Now your localhost port 5555 is forwarded to port 22 of the virtual machine.

3 The network-driver

To access the hardware, your final program should be *executed* by the **network-driver**. For this, you call

```
$ network-driver IFC1 ... IFCn - switch ARGS
```

where “IFC1 ... IFCn” is the list of interface names that you want **network-driver** to support (i.e. “lan0”, “lan1”) and “PROG” is the name of your binary and “ARGS” are the command-line arguments to “PROG”. Note the “.” (single minus) between the last interface name and “PROG”. Also, “PROG” must be given with the path or be in the PATH.

Once you start **switch** like this, you can read Ethernet frames and end-user commands from “**stdin**” and write Ethernet frames (and end-user output) to “**stdout**”.

Note that you must follow the **network-driver**’s particular format for inter-process communication when reading and writing. You will **not** be communicating directly with the console!

3.1 Build the driver

To compile the code, run:

```
# This requires gcc
$ make
# Creating network interfaces requires 'root' rights
$ sudo chmod +s network-driver
# Try it out:
$ ./network-driver eth0 - ./parser
```

Press CTRL-C to stop the **network-driver** and **parser**.

3.2 Understanding the driver

The output of the driver is always in binary and generally in network byte order. You can use a tool like **hexer** to make the output slightly more readable.

The driver will always output a series of messages starting with a **struct GLAB_MessageHeader** that includes a type and a size.

When the driver starts, it first writes a control message (of type 0) with payload that includes 6 bytes for each of the local interface’s MAC addresses to your **stdin**. Henceforce, messages received of type 0 will be single lines of command-line input (including the ‘n’-terminator, but excluding the 0-terminator of C) as typed in by the user.

Furthermore, the driver will output a **struct GLAB_MessageHeader** for each frame received. The **struct GLAB_MessageHeader** will be followed by the actual network frame, starting with the Ethernet frame excluding preamble, delimiter and FCS. The **struct GLAB_MessageHeader** includes the total length of the subsequent frame (encoded in network byte order, the size includes the **struct GLAB_MessageHeader**). The fixed message type identifies the number of the network interface, counting from one (also in network byte order).

In addition to writing received frames to your **stdin**, the driver also tries to read from your **stdout**. Applications must send the same message format to **stdout** that the driver sends them on **stdin**. The driver does **not** check that the source MAC is set correctly!

To write to the console’s **stdout**, use a message type of 0. You may directly write to **stderr** for error messages.

4 Provided code

You are given a few C snippets as starting points. However, these mostly serve to *illustrate* how to process the output from the driver. You are completely free to implement your application in *any* programming language. Note that each file includes about 20 LOC of a licensing statement, so the functions provided should not provide a significant advantage for implementations in C.

sample-parser.c This file includes a simple starting point for a wireshark-like frame inspection code. It mostly shows how the frames are received and a bit how to use the other C files. (82 LOC)

glab.h A struct defining a MAC Address and a few common C includes. (90 LOC)

print.c This file shows how to wrap messages to print them via the driver. (112 LOC)

loop.c This could be the main loop of your application. Includes some basic logic to look at each frame, decide whether it is the MACs, control or an Ethernet frame and then calls the respective function. (93 LOC)

crc.c An implementation of checksum algorithms. (194 LOC)

If you are using another programming language, you are free to re-use an existing CRC implementation in that language.

The main file for the exercise is **switch.c**. In this file, you should implement a program **switch** which forwards frames received on any interface to any other interface, but passively learns MAC addresses and optimizes subsequent traffic.

5 Warm-up exercises

These exercises may help you get familiar with the provided environment. They are **not graded**.

5.1 Frame parsing

Extend the simple frame **parser** to:

1. Output your system's MAC address(es) in the canonical human-readable format.
2. Output the source MAC, destination MAC, payload type and payload length of each frame received. Confirm your results with **wireshark**.

5.2 Implement a Hub

Implement **hub** which forwards frames received on any interface to all other interfaces (**eth0** through **eth3**), without changing them at all. The **hub** binary should take the list of interfaces to use on the command line.

6 Bonus exercise

This exercise may help you better understand the material from the networking course. It is not expected to be achievable within the period of the sprint and best done as a follow-up. It is **not graded**.

Implement *vswitch* which forwards frames received on any interface to any other interface, passively learns MAC addresses, and respects VLAN tags. As before, the command-line specifies the list of network interfaces you should switch on, but with additional options to specify the VLANs. Example:

```
$ network-driver eth0 eth1 eth2 eth3 - \  
  vswitch eth0[T:1,2] eth1[U:1] eth2[U:2] eth3[U:2]
```

This is supposed to run VLANs 1 and 2 tagged on **eth0**, and VLANs 1, 2 or 2 untagged on **eth1**, **eth2**, or **eth3** respectively. Network interfaces specified without “[]” should operate untagged on VLAN 0. It is not allowed to have interfaces accept both tagged and untagged frames.

Test your implementation against the Netgear switch of the lab. Bridge a tagged VLAN (**VID**= 3) from the Netgear switch (**eth1**) with two untagged notebooks (**eth2**, **eth3**) using the BananaPi.

Properties to consider for a virtual switch:

- Correct forwarding? Do frames flow bidirectionally between **eth2** and **eth3**?
- Correct switching? Is learning correctly implemented?
- Are VLAN tags added/stripped?
- Are correct limitations imposed on VLANs? Is say *VID* = 4 not forwarded to **eth2/eth3**?